

	UNIVERSIDAD AUTÓNOMA DE OCCIDENTE	PRIMER PARCIAL	Valoración: 5.0
	Facultad de Ingeniería	Asignatura: Servicios Telemáticos	Fecha: septiembre 8 de 2026
	Programa: Ing. Informática / Ing. de Datos e IA	Código: _____	Duración: _____
	Nombre: _____	Puntaje: _____	

PRIMER PARCIAL — SERVICIOS TELEMÁTICOS

Microproyecto integrador: DNS tolerante a fallos · Optimización de tráfico web · Publicación segura

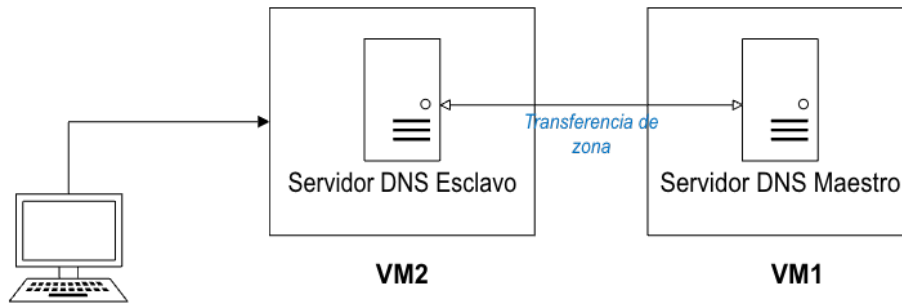
Instrucciones generales y entregables

Este parcial es un microproyecto práctico integrador. Las tres partes se construyen sobre una misma infraestructura virtualizada (VirtualBox / Vagrant). No se entrega informe escrito: el trabajo se sustenta y demuestra en vivo, y los scripts y archivos de configuración resultantes se suben a GitHub.

- **Modalidad:** trabajo en grupo (grupos ya conformados). Todos los integrantes deben poder explicar y ejecutar cualquier parte durante la sustentación.
- **Entregable en GitHub:** un repositorio con los archivos de configuración y scripts resultantes (named.conf, archivos de zona, configuración de Apache, páginas de prueba, scripts de medición y los comandos utilizados), organizados por parte.
- **Reproducibilidad:** el contenido del repositorio debe permitir volver a levantar la solución (p. ej. Vagrantfile y scripts de aprovisionamiento).
- **Sustentación y evidencia en vivo:** la evaluación es una demostración en tiempo real. El docente podrá pedir a cualquier integrante ejecutar comandos y mostrar los resultados en el momento; no se aceptan capturas ni informes como reemplazo de la demostración.
- **Integridad académica:** el uso de asistentes de IA debe declararse y el grupo debe poder explicar cada línea de configuración entregada.

Escenario

La empresa empresa.local requiere una infraestructura DNS tolerante a fallos, segura y auditable. Debe implementar un servidor DNS maestro y uno esclavo que permitan la transferencia de zona autenticada, la resolución directa e inversa, y la continuidad del servicio ante la caída del maestro, en un entorno simulado.



Requerimientos

1. **Instale y configure BIND9** en dos máquinas virtuales:
 - **VM1:** servidor DNS maestro — **maestro.empresa.local**.
 - **VM2:** servidor DNS esclavo — **esclavo.empresa.local**.
2. Configure en el maestro la zona directa empresa.local con, al menos, registros:
 - **A y AAAA** (IPv4 e IPv6) para **www**, **mail** y **ns1/ns2**.
 - **CNAME** (p. ej. **ftp** → **www**), **MX** y **NS**.
 - **SOA** coherente: use el patrón de serial **AAAAMMDDNN** y justifique los tiempos **refresh/retry/expire/minimum**.
3. **Configure la zona de resolución inversa** para la red local (ej. 50.168.192.in-addr.arpa) con los registros PTR correspondientes.
4. **Habilite NOTIFY** en el maestro y verifique que el esclavo obtiene la zona por transferencia (**AXFR**) y actualizaciones incrementales (**IXFR**).
5. **Transferencia SEGURA de zona con TSIG:** genere una clave simétrica (**tsig-keygen**) y restrinja la transferencia únicamente al esclavo autenticado con esa clave. La transferencia abierta a cualquier cliente no se acepta.
6. **Sincronización automática:** modifique un registro e incremente el serial en el maestro; demuestre que el esclavo se actualiza automáticamente sin intervención manual.
7. **Endurecimiento (hardening):**
 - **Desactive la recursión** (recursion no;). Un DNS autoritativo no debe comportarse como open resolver (riesgo de amplificación/DDoS): no debe resolver dominios externos como google.com.
 - **Restrinja allow-transfer** a la clave TSIG del esclavo y bloquee AXFR no autorizado.
 - **Limite allow-query** a las redes previstas y active **rate-limiting** (Response Rate Limiting) como mitigación adicional.
8. **Auditoría:** configure logging de las categorías queries, transfers y security en archivos separados, y muestre las entradas generadas durante las pruebas (el escenario exige que sea auditable).

9. **Continuidad:** configure una máquina cliente apuntando al esclavo, apague el maestro y demuestre que el cliente sigue resolviendo nombres directos e inversos.

Pruebas y verificación sugeridas

Validación de sintaxis y zonas (antes de recargar):

```
named-checkconf /etc/bind/named.conf
named-checkzone empresa.local /etc/bind/db.empresa.local
named-checkzone 50.168.192.in-addr.arpa /etc/bind/db.192.168.50
```

Resolución directa, inversa y transferencia autenticada:

```
dig @192.168.50.2 www.empresa.local A +short
dig @192.168.50.2 www.empresa.local AAAA +short
dig @192.168.50.3 -x 192.168.50.10 +short          # PTR desde el esclavo
dig @192.168.50.2 empresa.local AXFR -k /etc/bind/keys/esclavo.key
dig @192.168.50.2 google.com +short               # debe FALLAR (recursion no)
```

Nota: Para la prueba de continuidad, apague el maestro (systemctl stop bind9 o el equipo VM1) y repita las consultas dig contra el esclavo (VM2): deben seguir respondiendo mientras no expire el tiempo de expiry del SOA.

Entregables — Parte 1

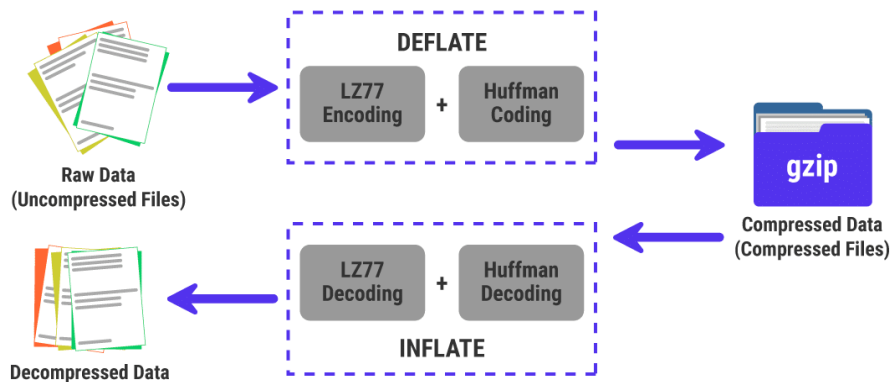
- En GitHub: named.conf y archivos de zona (directa e inversa) del maestro y del esclavo, comentados.
- Clave TSIG generada y fragmento de configuración que restringe la transferencia.
- En la sustentación (en vivo): transferencia exitosa, AXFR bloqueado sin clave, recursión denegada, resolución tras caída del maestro y las entradas de los logs de auditoría.

Rúbrica — Parte 1 (2.0 pts)

Criterio de evaluación	Puntos
Maestro con zona directa (A, AAAA, CNAME, MX, NS) y SOA correcto	0.4
Resolución inversa (PTR) funcionando	0.3
Esclavo con NOTIFY + transferencia (AXFR/IXFR) y sincronización automática	0.4
Transferencia segura con TSIG y bloqueo de AXFR no autorizado	0.3
Hardening: recursión desactivada, allow-query y RRL	0.2
Auditoría (logging) evidenciada	0.2
Continuidad del servicio con el maestro apagado	0.2
Total	2.0

Contexto

Uno de los aspectos clave para optimizar el rendimiento de un servidor web es la compresión de los recursos antes de transmitirlos al cliente. Apache implementa esta función principalmente con dos módulos: mod_deflate, que usa el algoritmo gzip (DEFLATE = LZ77 + Huffman), y mod_brotli, que usa el algoritmo Brotli de Google, generalmente con mejores ratios sobre texto. En esta parte no solo habilitará la compresión, sino que la caracterizará experimentalmente.



Requerimientos

1. **Instale y configure Apache** en una máquina virtual y publique un **sitio de prueba** con recursos representativos (ver tabla de tipos de archivo más abajo).
2. **Configure un DNS local** (reutilice la Parte 1 o un /etc/hosts) para resolver el dominio **parcial.empresa.local** hacia el servidor.

A) mod_deflate y niveles de compresión (DeflateCompressionLevel)

3. **Habilite mod_deflate** (a2enmod deflate) y aplique la compresión por tipo MIME a contenido de texto (HTML, CSS, JS, JSON, XML, SVG, texto plano).
4. **Evalúe al menos tres niveles** de compresión mediante la directiva **DeflateCompressionLevel** (rango 1–9): **bajo (1)**, **medio (6, por defecto)** y **máximo (9)**. Para cada nivel mida el tamaño resultante y el tiempo/costo de CPU.

```
# /etc/apache2/mods-available/deflate.conf (o VirtualHost)
<IfModule mod_deflate.c>
    DeflateCompressionLevel 6      # pruebe con 1, 6 y 9
    AddOutputFilterByType DEFLATE text/html text/plain text/xml \
        text/css application/javascript application/json image/svg+xml

    # Excluir binarios ya comprimidos (imágenes, video, zip)
    SetEnvIfNoCase Request_URI \.(?:gif|jpe?g|png|webp|mp4|avi|zip|gz|pdf)$ \
        no-gzip dont-vary
</IfModule>
```

5. **Excluya de la compresión** los tipos ya comprimidos (imágenes, video, archivos empaquetados) y justifique por qué comprimirlos es contraproducente.

B) mod_brotli (comparación frente a mod_deflate)

6. **Habilite mod_brotli** (a2enmod brotli) y configure la compresión Brotli para los mismos tipos de texto. Evalúe al menos dos niveles de calidad con **BrotliCompressionQuality** (rango 0–11), p. ej. 5 y 11.

```
# /etc/apache2/mods-available/brotli.conf (o VirtualHost)
<IfModule mod_brotli.c>
    BrotliCompressionQuality 11    # pruebe con 5 y 11
    BrotliCompressionWindow 22
    AddOutputFilterByType BROTLI_COMPRESS text/html text/plain text/xml \
        text/css application/javascript application/json image/svg+xml
</IfModule>
```

Nota: Los navegadores anuncian Accept-Encoding: br principalmente sobre conexiones seguras, por lo que la comparación de esta parte se hace por terminal: fuerce cada algoritmo con `curl -H "Accept-Encoding: br" o -H "Accept-Encoding: gzip"`. Cuando conviven ambos módulos y el cliente soporta br, Apache prioriza Brotli.

C) Tipos de archivo a evaluar

Publique y mida, como mínimo, los siguientes recursos. El objetivo es mostrar cómo varía el beneficio de la compresión según el tipo de contenido:

Tipo	Ejemplo de recurso	Comportamiento esperado
HTML	index.html (≈ 100–500 KB)	Alta compresibilidad
CSS	estilos.css minificado y sin minificar	Alta compresibilidad
JavaScript	app.js (biblioteca real)	Alta compresibilidad
JSON	datos.json (dataset grande)	Muy alta compresibilidad
SVG / XML	grafico.svg, feed.xml	Alta compresibilidad
Texto plano	lorem.txt (> 1 MB)	Muy alta compresibilidad
Imagen JPEG/PNG	foto.jpg, imagen.png	Nulo o negativo (excluir)
Video/binario	clip.mp4, paquete.zip	Nulo o negativo (excluir)

Metodología de medición

Para cada combinación (tipo de archivo × algoritmo × nivel) mida el tamaño transferido y el tiempo total. Registre también el tamaño sin comprimir (identity) como línea base.

```
# Tamaño y tiempo SIN comprimir (línea base)
curl -s -H 'Accept-Encoding: identity' -o /dev/null \
    -w 'sin=%{size_download}B t=%{time_total}s\n' http://parcial.empresa.local/app.js

# Con gzip (mod_deflate) – repita cambiando DeflateCompressionLevel 1/6/9
curl -s -H 'Accept-Encoding: gzip' -o /dev/null \
    -w 'gzip=%{size_download}B t=%{time_total}s\n' http://parcial.empresa.local/app.js

# Con brotli (mod_brotli) – repita con BrotliCompressionQuality 5/11
curl -s -H 'Accept-Encoding: br' -o /dev/null \
    -w 'br=%{size_download}B t=%{time_total}s\n' http://parcial.empresa.local/app.js

# Verificar el encabezado de codificación aplicado
curl -s -H 'Accept-Encoding: br,gzip' -I http://parcial.empresa.local/app.js | grep -i content-encoding
```

Muestre además en vivo, desde el navegador, el encabezado de codificación (F12 → pestaña Network → columna Content-Encoding y tamaño transferido) y el tráfico comprimido con **Wireshark** (filtro http).

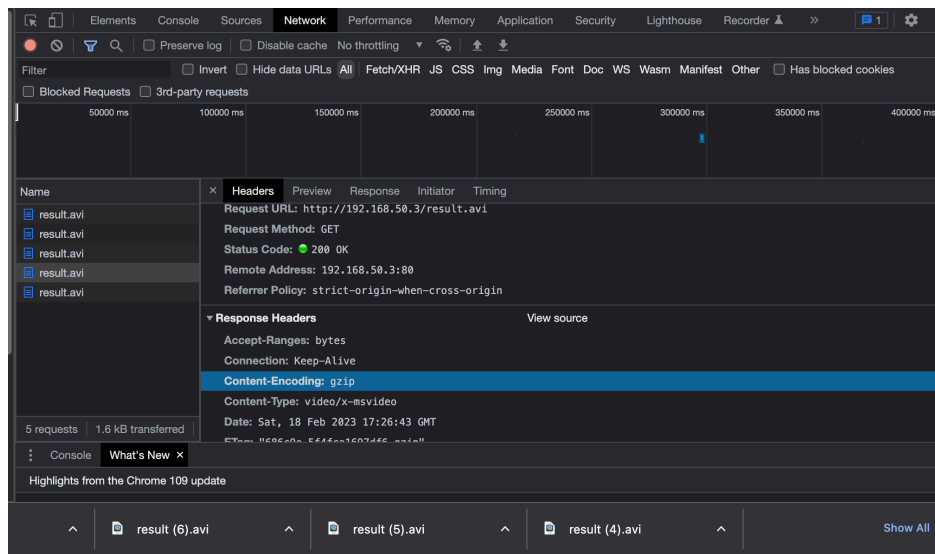


Tabla comparativa de resultados (a diligenciar)

Complete una tabla como la siguiente por cada tipo de archivo relevante. El ratio de compresión = tamaño comprimido / tamaño original (menor es mejor); el ahorro % = $(1 - \text{ratio}) \times 100$.

Algoritmo / nivel	Tamaño	Ratio	Ahorro %	Tiempo / CPU
Sin comprimir (base)				
gzip nivel 1				
gzip nivel 6				
gzip nivel 9				
brotli calidad 5				
brotli calidad 11				

Recurso evaluado: _____ Tamaño original: _____

Análisis crítico (obligatorio)

Con base en TODAS las mediciones anteriores, redacte un análisis que responda de forma argumentada y con datos:

- Brotli vs gzip:** ¿cuánto mejora Brotli el ratio frente a gzip sobre texto? ¿En qué tipos de archivo la diferencia es significativa y en cuáles es marginal?
- Niveles de compresión:** ¿qué gana en tamaño al pasar de nivel 1 → 6 → 9 (gzip) y 5 → 11 (brotli)? ¿Justifica ese ahorro adicional el mayor costo de CPU/latencia? Identifique el punto de rendimientos decrecientes.
- Tipos de archivo:** ¿por qué los recursos ya comprimidos (JPEG, PNG, MP4, ZIP) no se benefician —o incluso crecen— al comprimirlos? Sustente con sus datos.
- Impacto en CPU y ancho de banda:** discuta el equilibrio entre ahorro de ancho de banda y consumo de CPU del servidor, y cómo cambia bajo concurrencia alta.
- Contenido estático vs dinámico:** argumente cuándo conviene precomprimir en disco (brotli 11 servido con mod_headers) frente a comprimir al vuelo, y proponga una configuración recomendada para producción justificando algoritmo y nivel por tipo de contenido.

Entregables — Parte 2

- En GitHub: configuración de mod_deflate (con los tres niveles), de mod_brotli, el sitio de prueba y los scripts de medición.
- En la sustentación (en vivo): Content-Encoding y tamaños con curl, navegador (Network) y Wireshark.
- Tabla comparativa diligenciada (una por tipo de archivo), incluida en el repositorio como archivo de datos (p. ej. CSV o Markdown).
- Análisis crítico de los cinco puntos anteriores, argumentado con datos durante la sustentación.

Rúbrica — Parte 2 (2.0 pts)

Criterio de evaluación	Puntos
Apache + mod_deflate funcionando y DNS del dominio parcial.empresa.local	0.3
Evaluación de los tres niveles DeflateCompressionLevel (1/6/9) con mediciones	0.3
mod_brotli habilitado y comparado (calidad 5 y 11)	0.3
Pruebas sobre distintos tipos de archivo + exclusión correcta de binarios	0.3
Evidencia en vivo con curl, navegador (Network) y Wireshark	0.3
Tabla comparativa completa (ratio, ahorro %, tiempo/CPU)	0.2
Análisis crítico argumentado con datos (5 puntos)	0.3
Total	2.0

Objetivo

Publique en Internet el servidor web de la Parte 2 mediante un túnel seguro, de modo que sus recursos — incluida la compresión configurada— sean accesibles desde fuera de la red local. Debe agregar una página personalizada que permita identificar su despliegue.

Requerimientos

1. **Servidor web en funcionamiento:** reutilice el sitio de la Parte 2 (con compresión habilitada) accesible desde el anfitrión.
2. **Página personalizada:** cree una página (p. ej. pagina_personalizada.html) con contenido único (su nombre, código, fecha y un identificador) para verificar el acceso remoto.
3. **Túnel seguro:** instale y configure una de las siguientes opciones y agréguela al PATH:
 - **ngrok** (versión gratuita): ngrok http 80 genera una URL pública que redirige a su servidor local.
 - **cloudflared** (Cloudflare Tunnel, alternativa recomendada y sin límite de sesión): cloudflared tunnel --url http://localhost:80.
 - **vagrant share** como alternativa si trabaja con Vagrant.

```
# Opción A – ngrok
ngrok http 80
# → Forwarding https://abcd1234.ngrok-free.app -> http://localhost:80
```

```
# Opción B – Cloudflare Tunnel (sin registro para pruebas rápidas)
cloudflared tunnel --url http://localhost:80
```

4. **Prueba de acceso remoto:** desde un dispositivo en otra red (datos móviles u otra conexión), acceda a la URL pública y verifique que se muestra su página personalizada.
5. **Verifique que la compresión sobrevive al túnel:** ejecute curl contra la URL pública y confirme que el encabezado Content-Encoding (gzip o br) sigue presente.

```
curl -s -H 'Accept-Encoding: br,gzip' -I https://abcd1234.ngrok-free.app/app.js | grep -i content-encoding
```

6. **Análisis de seguridad (breve):** discuta los riesgos de exponer un servicio con un túnel público (superficie de exposición, ausencia de autenticación, límites del plan gratuito) y al menos dos mitigaciones (autenticación básica, restricción por IP, apagar el túnel al terminar).

Nota: Los túneles públicos exponen su equipo a Internet mientras están activos. No deje el túnel abierto más allá de la prueba y no publique servicios con datos sensibles.

Entregables — Parte 3

- Página personalizada (código en GitHub) y demostración en vivo del acceso remoto desde otra red mostrando la URL pública.
- En vivo: túnel activo (salida de ngrok/cloudflared) y el encabezado de compresión a través del túnel.
- Breve análisis de seguridad con las mitigaciones propuestas, argumentado en la sustentación.

Rúbrica — Parte 3 (1.0 pt)

Criterio de evaluación	Puntos
Túnel activo y URL pública funcionando	0.4
Página personalizada visible en el acceso remoto desde otra red	0.3

Criterio de evaluación	Puntos
Compresión verificada a través del túnel	0.1
Análisis de seguridad con al menos dos mitigaciones	0.2
Total	1.0

Consolidado de evaluación

Valor	Descripción	Obtenido	Observaciones
2.0	Parte 1 — DNS Maestro/Esclavo (transferencia segura TSIG, inversa, hardening, auditoría, continuidad)		
2.0	Parte 2 — Compresión en Apache (mod_deflate por niveles, mod_brotli, tipos de archivo y análisis)		
1.0	Parte 3 — Túnel seguro (ngrok/cloudflared) con página personalizada y análisis de seguridad		
5.0	TOTAL		

Oscar H. Mondragón, Ph.D. — Servicios Telemáticos · Facultad de Ingeniería · Universidad Autónoma de Occidente